TECHNICAL REVIEW

MESI / MOESI Cache Coherence

Protocols, state machines, verified claims, an executable C++ verification model with test cases, benchmarks, and extensions in shared-memory multiprocessors

Summary

MESI is the standard four-state baseline for cache coherence. MOESI adds an Owned state so dirty data can be shared more efficiently, and later variants trade complexity for lower traffic, better predictability, or lower energy. Sections 1 to 5 organize the literature evidence: the core explanation of the problem and its two main mechanisms, a comparison of protocol states, twelve true/false verified cases, a read-miss and write walkthrough, and a survey of performance results and extensions. Sections 6 to 8 and the appendices are new in this edition. They document an executable C++ model of MESI and MOESI with a runtime invariant checker, the 42 test cases it was run against, and benchmark results comparing the two protocols, together with a description of the code design and working flow.

#	Section	Contents
1	Core explanation	Why coherence is needed; snooping vs. directory
2	State comparison	MESI, MOESI, MESIF, MOESIF, PMESI
3	Verified cases	Twelve true/false claims with evidence
4	Read-miss walkthrough	Install state and later write behavior
5	Performance and extensions	Measured results and protocol variants
6	Implementation (new)	C++ model, transition rules, invariants I1 to I5
7	Test cases and results (new)	19 directed, 14 randomized and 9 fault-injection tests; state trace; cross-check of Section 3
8	Benchmarks (new)	Six workloads, core-count scaling, checker overhead, charts
9	Conclusion	Key takeaways
A, B	Appendices (new)	Code design and working flow; raw benchmark data
	References	Sources cited

Citations appear inline as author and year and are listed with links in the References section. Sections 6 to 8 and the appendices report original implementation and measurement work; their numbers come from the code described in Appendix A and the data in Appendix B, not from the cited literature.

1. Core explanation

Cache coherence is the rule set that keeps copies of shared data consistent across multiple caches and main memory in shared-memory multiprocessors. (Ivanov 2001; Kaur 2018; Kehagias 2016) The problem arises because modern multi-core chips give each core a private cache while all cores still share one main memory. (Faeq 2021; Amory 2021; Kehagias 2016) If one core modifies a value in its cache while another core keeps an older copy, the system can return stale data unless a coherence protocol updates or invalidates the other copies. (Faeq 2021; Tiwari 2014; Krishna 2025)

Snooping and directory-based mechanisms

Two broad coherence mechanisms are described in the literature: snooping and directory-based coherence. (Blanchet 2013; Tiwari 2014; Somarouthu 2025) Snooping protocols make each cache watch a common bus for reads and writes from other caches. (Blanchet 2013; Faeq 2021; Krishna 2025) Directory protocols centralize sharing information in a table or directory and are used to track sharers and serve misses at larger scale. (Blanchet 2013; Komuravelli 2014)

Aspect	Snooping	Directory-based
Sharing information	Distributed; each cache watches a common bus	Centralized in a table or directory
How misses are served	Other caches observe and react to bus traffic	Directory tracks sharers and directs the miss
Typical scale	Smaller systems sharing one bus	Larger scale

Table 1. Snooping versus directory-based coherence, as described in the cited literature.

Cache access flow

Figure 1 shows the general path of a memory access. A hit proceeds to the read or write path, and a write forces an invalidate or upgrade before the line is used. A miss checks the bus or directory, fetches the data, and assigns a coherence state before the line is used.

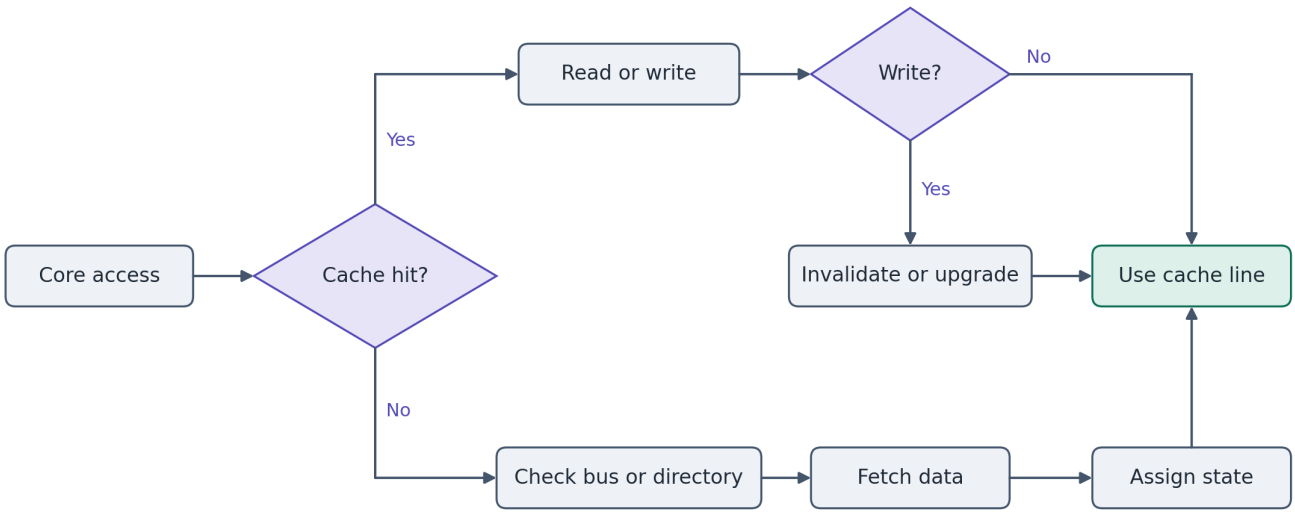


Figure 1. Core access flow with cache hit, write, and miss paths.

2. State comparison

The protocols below differ mainly in the states they define and the optimization each state enables.

Protocol	States	Key optimization	Evidence
MESI	M, E, S, I	Exclusive avoids invalidation on write hits	Komuravelli 2014; Somarouthu 2025
MOESI	M, O, E, S, I	Owned shares dirty data without immediate write-back	Alkhamisi 2022; Somarouthu 2025
MESIF	M, E, S, I, F	Reduces responders on cache miss	Somarouthu 2025; Patil 2019
MOESIF	M, O, E, S, I, F	Improves bandwidth, energy, and access time	Patil 2019
PMESI	Predictable MESI variant	Derives worst-case latency bounds	Kaushik 2021

Table 2. Coherence protocol states, key optimizations, and supporting evidence.

State meanings

State	Meaning in this review
Modified (M)	Cache holds a changed copy of the line.
Owned (O)	MOESI only. Shares dirty data without an immediate write-back to memory.
Exclusive (E)	Only this cache has the line; a later write needs no invalidation of others.
Shared (S)	Multiple clean copies may exist in different caches.
Invalid (I)	Cache does not hold a usable copy.
Forward (F)	MESIF and MOESIF. Reduces the number of responders on a miss.

Table 3. Summary of states used across the compared protocols.

3. Verified cases

Each case below gives a true/false statement and the evidence-backed explanation.

#	Verdict	Statement	Sources
1	TRUE	MESI has four states: Modified, Exclusive, Shared, and Invalid.	Komuravelli 2014; Faeq 2021; Alkhamisi 2022
2	FALSE	MESI includes an Owned state. Owned belongs to MOESI, not MESI.	Komuravelli 2014; Somarouthu 2025; Dey 2014; Alkhamisi 2022
3	TRUE	Exclusive lets a core write later without invalidating others if no other cache has the line.	Komuravelli 2014; Somarouthu 2025; Faeq 2021
4	FALSE	Shared means no other cache can have the line. Shared explicitly allows multiple clean copies.	Somarouthu 2025; Komuravelli 2014
5	TRUE	Snooping works by monitoring a shared bus.	Blanchet 2013; Krishna 2025; Faeq 2021
6	TRUE	Directory protocols track sharers in a centralized structure.	Blanchet 2013; Komuravelli 2014
7	FALSE	MOESI always writes modified shared data back to memory before another core can read it.	Somarouthu 2025; Alkhamisi 2022
8	TRUE	MOESI reduced load misses by 48% and memory latency by 35% in one dual-processor simulation.	Kaur 2018
9	TRUE	That same study reported 18% lower power, 34% higher gate count, and 22% better execution time for MOESI over MESI/MSI.	Kaur 2018
10	TRUE	Formal verification found six bugs in a state-of-the-art MESI implementation.	Komuravelli 2014
11	TRUE	MESI had 15x more reachable states and 20x longer model-checking time than DeNovo in that verification study.	Komuravelli 2014
12	FALSE	Coherence protocols are only relevant to two-core systems.	Tiwari 2014; Fotouhi 2025; Kaushik 2021

Table 4. Twelve verified true/false cases.

Reading the results

Cases 1 to 4 pin down the state definitions. Owned is a MOESI addition, Exclusive removes an invalidation on a later write, and Shared allows several clean copies. Cases 5 and 6 restate the two mechanisms. Cases 7 to 9 concern MOESI, whose dirty sharing avoids always writing back before a read, and whose reported gains in one dual-processor study came with a higher gate count. Cases 10 and 11 show that verification effort is itself a cost of protocol complexity. Case 12 rules out the idea that coherence matters only for two cores.

Update in this edition: cases 1 to 4, 7 and 12 can also be exercised directly on the executable model of Section 6; Table 14 in Section 7 records which cases the simulator confirms and which it cannot speak to.

4. Read-miss walkthrough

Figure 2 follows a read miss through to a later write. If no other cache holds the line, it is loaded from memory. If another cache holds a clean copy, the line is installed as Shared or Exclusive. In MOESI, if another cache holds the line dirty, the owner supplies the data. The state installed then determines what a later write must do: an Exclusive line moves silently to Modified, a Shared line requires invalidating the other copies, and an Owned line upgrades ownership.

Section 7.4 replays this walkthrough step by step on the executable model and prints the state of every cache after each access.

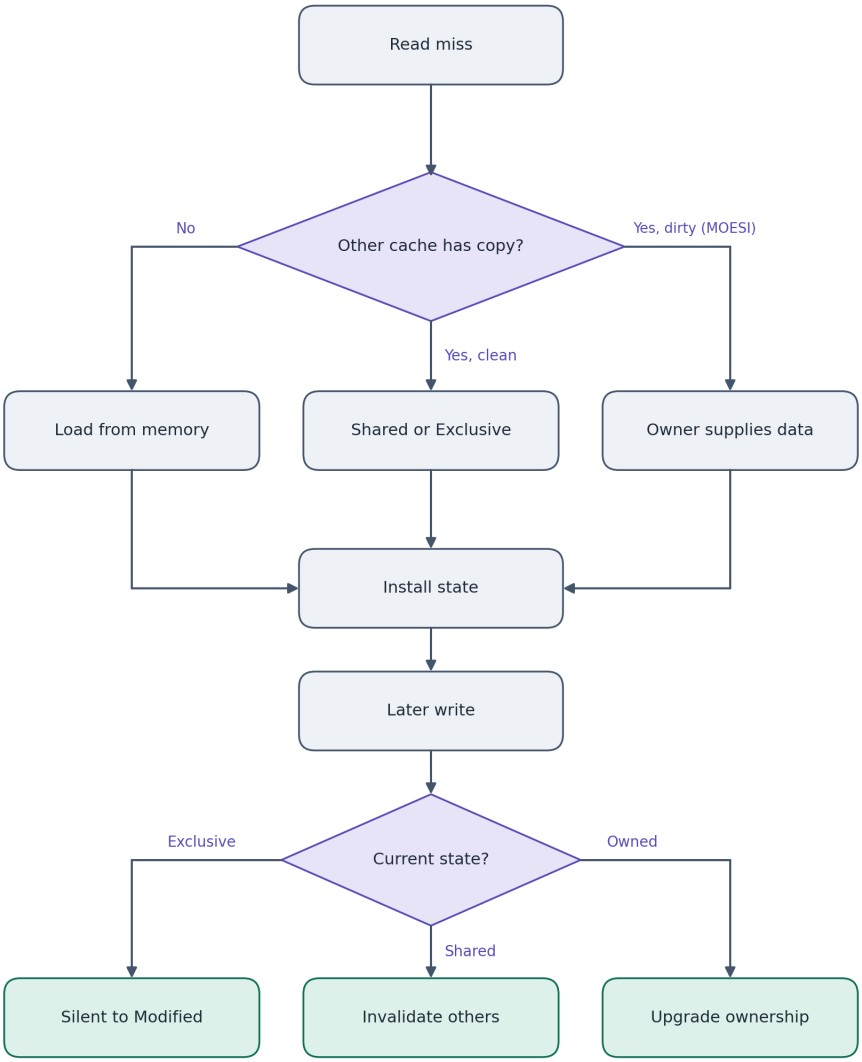


Figure 2. Read-miss handling and later write behavior by state.

5. Performance and extensions

Performance depends strongly on the chosen coherence protocol. (Kaur 2018; Altwaijry 2013; Krishna 2025) In one comparison, performance improved moving across MI, MESI two-level, MESI three-level, MOESI, and MOESI token configurations, and adding an L3 cache improved MESI three-level over MESI two-level. (Tiwari 2014) Another study reported that MESI consumed more power than MSI and MOESI in a snooping dual-core design. (Krishna 2025)

Protocol extensions

Several extensions target specific limits of MESI or MOESI.

Extension	Reported result	Source
Improved-MOESI	Outperformed classic MOESI, MESI, and MSI in simulation.	Altwaijry 2013
MOESIF	Better bandwidth use by reducing write-backs and responders; average 4.47% energy and 9.21% access-time savings over MOESI.	Patil 2019
MOESIL	2.89% lower miss rate than MOESI for locked caches in mixed-criticality systems.	Nair 2021
MESI-A	Ignores coherence communication for approximate data; improved efficiency with 5% to 20% approximated data on PARSEC workloads.	Saraswat 2021
PMESI	Up to 4x better average-case performance than cache bypassing, but 1.46x slower average execution than conventional MESI.	Kaushik 2021

Table 5. Reported results for MESI and MOESI extensions.

Real-time and predictability

For real-time systems, predictable MESI variants derive worst-case latency bounds, but coherence latency still scales quadratically with core count. (Kaushik 2021) PMESI improved average-case performance by up to 4x over cache bypassing, while still slowing average execution by 1.46x versus conventional MESI. (Kaushik 2021)

Simulation and hardware validation

Educational and FPGA implementations also show that MESI state transitions can be demonstrated and validated in simulators and hardware test benches. (Faeq 2021; Amory 2021; Kehagias 2016) Sections 6 to 8 add a software model in this tradition, with a runtime checker and fault-injection tests.

6. Implementation: C++ verification model

To check the protocol behavior described above, the two protocols were implemented as an executable model in C++17 (`coherence_verify.cpp`) and driven by a test suite and a benchmark driver (`coherence_bench.cpp`). Appendix A describes the code design and working flow. The model is a functional, event-counting simulator of a snooping bus. It is not cycle-accurate, and it does not model latency, power or gate count.

6.1 Design

Component	Description
System	N private caches plus a shared memory on an atomic snooping bus. The protocol (MESI or MOESI) is a constructor argument. Each line holds one 32-bit word and the caches have unlimited capacity, so lines leave only through an explicit `evict()`. Operations: `read`, `write`, `evict`.
Stats	Counters for hits, misses, BusRd, BusRdX, BusUpgr, memory writes, cache-to-cache supplies and silent E to M upgrades.
Checker	`System::check(addr)` evaluates invariants I1 to I4 over all caches for one address (Table 8).
Harness	Wraps System with a sequential golden memory model. After every operation it runs the checker and compares every load with the latest stored value (I5); the first violation is recorded.
Fault injection	A `Bug` enum deliberately breaks one rule of the protocol so that the tests can confirm the checker detects it.

Table 6. Components of the model.

6.2 Transition rules as implemented

Event	MESI	MOESI
Read hit (S, E, O, M)	No bus transaction, no state change	Same
Read miss, no other copy	Install E from memory	Same
Read miss, another cache has E	That cache E -> S; requester installs S	Same
Read miss, others hold only S	Requester installs S; memory supplies	Same
Read miss, another cache has M	That cache M -> S and flushes to memory; requester installs S	That cache M -> O and supplies the data; memory is **not** updated; requester installs S
Read miss, another cache has O	Not applicable (no O state)	O stays O and supplies the data; requester installs S
Write hit in M	Stays M	Same
Write hit in E	Silent E -> M, no bus transaction	Same
Write hit in S or O	BusUpgr: invalidate all other copies, then M	Same
Write miss	BusRdX: invalidate all other copies, requester M	Same
Evict M	Write back to memory	Same
Evict O	Not applicable	Write back to memory; sharers remain S
Evict S or E	Silent	Silent

Table 7. State transitions implemented for each protocol.

6.3 Invariants checked after every operation

ID	Invariant	Meaning
I1	Single writer, multiple readers	If a line is M or E in one cache, no other cache holds a valid copy.
I2	Single owner	At most one cache holds the line in M, O or E. MESI never holds an O copy.
I3	Data agreement	All valid copies of a line hold identical data.
I4	Memory consistency	If no cache holds the line in M or O (no dirty copy), every valid copy equals memory.
I5	Golden-model check	Every load returns the value of the most recent store to that address.

Table 8. Invariants enforced by the checker.

6.4 Build and run

```
g++ -std=c++17 -O2 -Wall -Wextra coherence_verify.cpp -o coherence_verify && ./coherence_verify   # 42 tests
g++ -std=c++17 -O2 coherence_bench.cpp -o coherence_bench && ./coherence_bench > bench_results.csv
python3 plot_bench.py                                                                              # charts
```

Scope and limitations. The bus is atomic, so there are no transient states or races; verifying a split-transaction protocol would need a model checker such as Murphi or TLA+. The invariants are checked on the executed traces, not proven over all reachable states. Directory-based coherence and the F state of MESIF and MOESIF are not modeled.

7. Test cases and results

The suite contains 42 tests in four groups. Every test passed in the final run (42 of 42), and each directed or randomized test also required that no invariant I1 to I5 was violated at any point.

7.1 Directed tests

ID	Scenario	Expected outcome checked	Result
MESI-01	P0 reads a cold line	State E,I,I	PASS
MESI-02	P0 reads, then writes (E to M)	State M,I,I; no BusRdX or BusUpgr issued; one silent upgrade counted	PASS
MESI-03	P0 reads, then P1 reads	State S,S,I	PASS
MESI-04	Three cores read; P1 writes	State I,M,I; exactly one BusUpgr	PASS
MESI-05	P0 writes 42; P1 reads	State S,S,I; P1 gets 42; memory updated to 42	PASS
MESI-06	P0 writes 1; P1 writes 2 (write miss on remote M)	State I,M,I; P2 later reads 2	PASS
MESI-07	P0 reads, then P1 reads (E downgrade)	State S,S	PASS
MESI-08	P0 writes 99 and evicts; P1 reads and evicts	Writeback gives memory 99; clean eviction writes nothing; P0 re-reads 99	PASS
MESI-09	P0 writes; P1, P2 read	No cache ever enters O	PASS
MESI-10	Two cores write two different addresses	Independent states M,I and I,M; cross reads return the right data	PASS

Table 9. MESI directed tests (10). Cores are P0, P1, P2 in that order for the state vectors.

ID	Scenario	Expected outcome checked	Result
MOESI-01	P0 reads, then writes	E,I,I then M,I,I	PASS
MOESI-02	P0 writes 11; P1 reads	State O,S,I; P1 gets 11; memory still 0 (stale by design)	PASS
MOESI-03	P0 writes; P1, P2, P3 read	State O,S,S,S	PASS
MOESI-04	P0 writes, P1 and P2 read, P0 writes again	State M,I,I (sharers invalidated)	PASS
MOESI-05	P0 writes; P1 reads; P1 writes 8; P2 reads	I,M,I then I,O,S; P2 gets 8	PASS
MOESI-06	P0 writes 77; P1, P2 read; P0 evicts	State I,S,S; O writeback gives memory 77	PASS
MOESI-07	P0 writes; P1 reads; P1 evicts	State O,I,I; no memory write	PASS
MOESI-08	P0 writes; P1 reads; P2 writes (write miss on O)	State I,I,M	PASS
MOESI-09	100 producer/consumer hand-offs on MESI and on MOESI	MOESI performs fewer memory writes than MESI (0 versus 100)	PASS

Table 10. MOESI directed tests (9). Cores are P0, P1, P2 in that order for the state vectors.

7.2 Randomized stress tests

Random sequences use 45% reads, 40% writes (each with a unique value) and 15% evictions over a fixed pool of 64-byte-aligned addresses. The harness checks I1 to I5 after every operation.

Protocol	Configuration	Result
MESI	seed=1 4 cores x 6 addrs x 100k ops	**PASS**
MESI	seed=2 4 cores x 6 addrs x 100k ops	**PASS**
MESI	seed=3 4 cores x 6 addrs x 100k ops	**PASS**
MESI	seed=42 4 cores x 6 addrs x 100k ops	**PASS**
MESI	seed=2024 4 cores x 6 addrs x 100k ops	**PASS**
MESI	heavy contention: 8 cores x 1 addr x 100k ops	**PASS**
MESI	wide sharing: 16 cores x 32 addrs x 200k ops	**PASS**
MOESI	seed=1 4 cores x 6 addrs x 100k ops	**PASS**
MOESI	seed=2 4 cores x 6 addrs x 100k ops	**PASS**
MOESI	seed=3 4 cores x 6 addrs x 100k ops	**PASS**
MOESI	seed=42 4 cores x 6 addrs x 100k ops	**PASS**
MOESI	seed=2024 4 cores x 6 addrs x 100k ops	**PASS**
MOESI	heavy contention: 8 cores x 1 addr x 100k ops	**PASS**
MOESI	wide sharing: 16 cores x 32 addrs x 200k ops	**PASS**

Table 11. Randomized stress tests (14), 800,000 checked operations per protocol in total.

7.3 Fault-injection tests

A checker is only useful if it can fail. Each test runs the randomized workload (4 cores, 4 addresses, seed 5, up to 20,000 operations) on a protocol with one deliberately injected fault and passes only if the checker reports a violation.

Protocol	Injected fault	Description	Invariant	Ops	Result
MESI	SkipInvalidateOnUpgrade	A write to an S or O line does not invalidate the other copies	I3	39	**Detected**
MOESI	SkipInvalidateOnUpgrade	A write to an S or O line does not invalidate the other copies	I3	39	**Detected**
MESI	NoDowngradeEOnRead	An E copy stays E when another cache reads the line	I1	9	**Detected**
MOESI	NoDowngradeEOnRead	An E copy stays E when another cache reads the line	I1	9	**Detected**
MESI	NoFlushOnMtoS	On a remote read, M -> S without updating memory (MESI)	I4	7	**Detected**
MESI	LostWritebackOnEvict	Evicting an M or O line does not write it back	I5	75	**Detected**
MOESI	LostWritebackOnEvict	Evicting an M or O line does not write it back	I4	53	**Detected**
MESI	WriteMissNoInvalidate	A write miss does not invalidate the other copies	I3	8	**Detected**
MOESI	WriteMissNoInvalidate	A write miss does not invalidate the other copies	I3	8	**Detected**

Table 12. Fault-injection results (9 of 9 detected). Different faults are caught by different invariants: for example a lost writeback is caught by I5 (a stale load) in MESI but by I4 in MOESI, where sharers of the Owned line still hold data that memory lacks.

7.4 Replay of the Section 4 walkthrough

The same access sequence was run on both protocols (three cores, one address, initial memory 0). The columns show the state of P0, P1, P2 and the value held in memory after each step. The two protocols diverge exactly where Section 3 predicts: MESI flushes a dirty line to memory when it is shared, while MOESI keeps it in the Owned state and leaves memory stale until the owner is evicted.

Step	Operation	MESI states	MESI memory	MOESI states	MOESI memory
1	P0 reads (cold miss)	**E,I,I**	0	**E,I,I**	0
2	P1 reads	**S,S,I**	0	**S,S,I**	0
3	P1 writes 5	**I,M,I**	0	**I,M,I**	0
4	P2 reads	**I,S,S**	5	**I,O,S**	0
5	P2 writes 6	**I,I,M**	5	**I,I,M**	0
6	P0 reads	**S,I,S**	6	**S,I,O**	0
7	P2 evicts line	**S,I,I**	6	**S,I,I**	6

Table 13. State trace of the read-miss walkthrough. Steps 4 and 6 show the MESI flush (memory becomes 5 and 6) versus the MOESI Owned state (memory stays 0 until step 7).

7.5 Cross-check of the Section 3 cases

Table 14 records which of the twelve literature cases the executable model can confirm, and is explicit about those it cannot speak to.

Case	Model evidence	Status
1	Only M, E, S, I appear in all MESI runs; invariant I2 rejects any O copy (MESI-09, 800k checked random ops).	Confirmed
2	No Owned state is ever reached under MESI (MESI-09).	Confirmed (false claim)
3	E to M is a silent upgrade with no bus traffic (MESI-02; 'silent E to M' counter).	Confirmed
4	Two clean copies coexist in S,S (MESI-03; also S,S,S in MESI-04).	Confirmed (false claim)
5	The model is an atomic snooping bus by construction, so this is an assumption of the model rather than a result.	By construction
6	Directory coherence is not modeled.	Not modeled
7	After M -> O the memory value stays stale (MOESI-02: memory 0 while the owner holds 11), so dirty data is not always written back first.	Confirmed (false claim)
8	The model has no timing, so load-miss counts are the only comparable metric. Misses are identical for MESI and MOESI in every workload (Table 16); the 48% reduction and 35% latency figure are not reproduced.	Not reproducible
9	Power, gate count and execution time are not modeled.	Not modeled
10, 11	The model uses runtime invariants and fault injection on executed traces, not exhaustive state-space exploration, so bug counts and reachable-state ratios are not comparable.	Different method
12	The invariants held in runs with 2, 4, 8, 16 and 32 cores (Section 8).	Confirmed (false claim)

Table 14. Section 3 cases against the executable model.

8. Benchmarks

8.1 Method

The benchmark driver runs six synthetic access patterns on both protocols through the same checking harness, so every number below comes from a run in which invariants I1 to I5 held; any violation aborts the benchmark. Each workload executes 200,000 operations on 8 cores with seed 1. Core-count scaling uses 2, 4, 8, 16 and 32 cores. Throughput measures 2,000,000 operations (8 cores, mixed random) compiled with -O2, once with the raw simulator and once with the checker enabled.

Workload	Access pattern
producer_consumer	Core 0 writes one shared line, then the other cores each read it; repeated.
migratory	Cores take turns doing read-then-write on one shared line.
read_mostly	Random core and address (16 lines); 95% reads, 5% writes.
write_contention	Random core writes one shared line (ping-pong).
private_data	Each core touches only its own 8 lines; 50% reads, 50% writes.
mixed_random	8 lines; 45% reads, 40% writes, 15% evictions.

Table 15. Benchmark workloads.

8.2 Results

Workload	Protocol	Hit rate	Bus txns / 1k ops	Memory writes / 1k ops	Cache-to-cache / 1k ops
producer_consumer	MESI	12.5%	1000	125.0	125.0
	MOESI	12.5%	1000	0.0	875.0
migratory	MESI	50.0%	1000	500.0	500.0
	MOESI	50.0%	1000	0.0	500.0
read_mostly	MESI	73.9%	298	47.4	47.5
	MOESI	73.9%	298	0.0	247.5
write_contention	MESI	12.5%	875	0.0	0.0
	MOESI	12.5%	875	0.0	0.0
private_data	MESI	100.0%	0	0.0	0.0
	MOESI	100.0%	0	0.0	0.0
mixed_random	MESI	19.0%	724	202.5	195.6
	MOESI	19.0%	724	18.2	334.4

Table 16. Results per workload (8 cores, 200,000 operations each). Raw counters are in Appendix B.

Memory writes. MOESI removes the write-backs that MESI performs when a dirty line is shared: producer/consumer 125 to 0, migratory 500 to 0, read-mostly 47.4 to 0, and mixed random 202 to 18 writes per 1,000 operations. Write contention and private data show no difference because no dirty line is ever shared. The remaining MOESI writes in mixed random are the O or M evictions.

Cache-to-cache transfers. MOESI has more of them (producer/consumer 125 versus 875 per 1,000 operations), because the Owned cache supplies every later reader where MESI would send the read to memory. MOESI trades cache-to-cache bandwidth for memory bandwidth.

Hit rate and bus traffic. Both are identical for the two protocols in every workload: MOESI changes who supplies data and when memory is written, not which bus transactions occur.

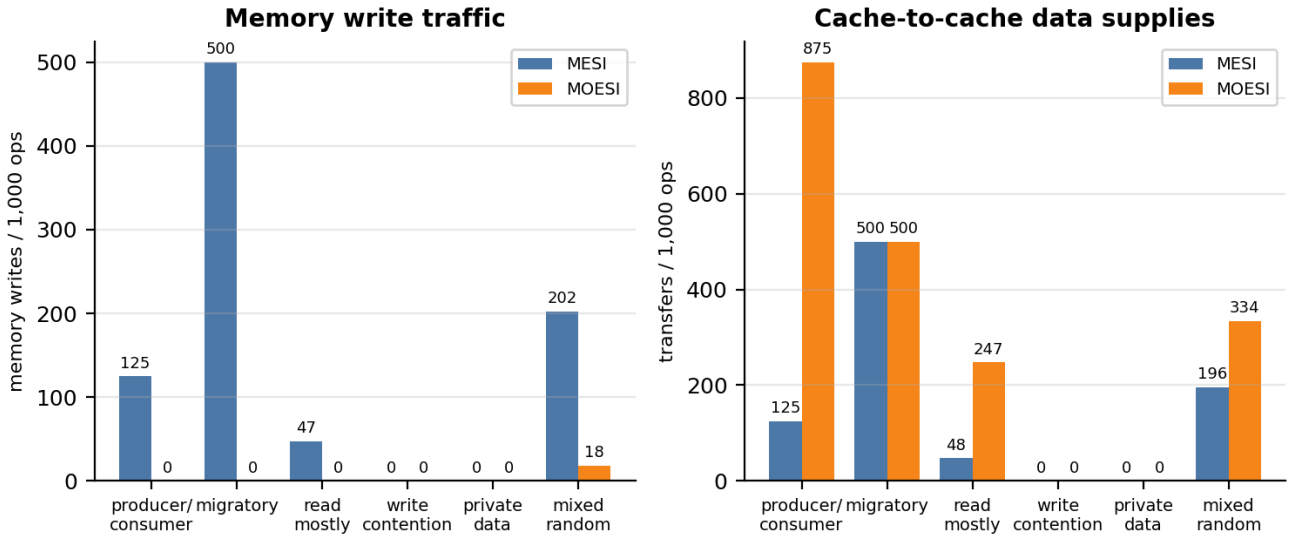


Figure 3. Memory write traffic and cache-to-cache supplies per 1,000 operations (8 cores).

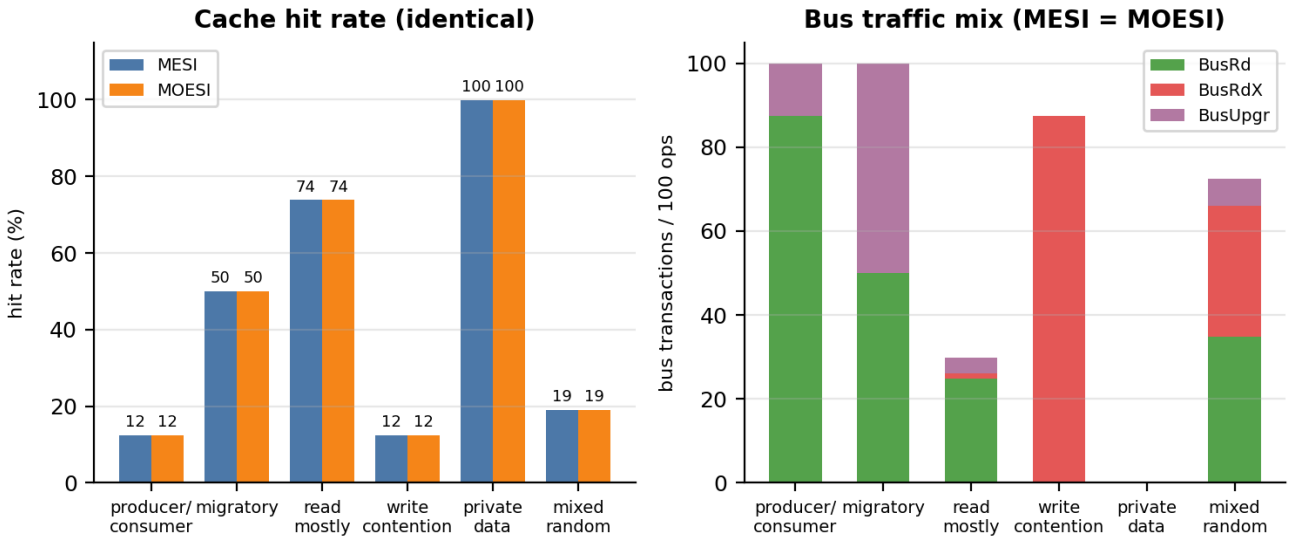


Figure 4. Cache hit rate and bus transaction mix per workload; the bus mix is the same for MESI and MOESI.

8.3 Scaling and checker overhead

Cores	MESI prod/cons	MOESI prod/cons	MESI mixed	MOESI mixed
2	500.0	0.0	170.2	63.3
4	250.0	0.0	192.7	34.5
8	125.0	0.0	202.5	18.2
16	62.5	0.0	207.2	9.3
32	31.2	0.0	209.2	4.7

Table 17. Memory writes per 1,000 operations versus core count (prod/cons = producer/consumer; mixed = mixed random).

In producer/consumer the MESI write-back rate falls as cores are added only because the writer's share of the operations shrinks; MOESI stays at zero. In mixed random MESI stays at roughly 170 to 210 writes per 1,000 operations from 2 to 32 cores, while MOESI falls from about 63 to about 5. The invariants held at every core count.

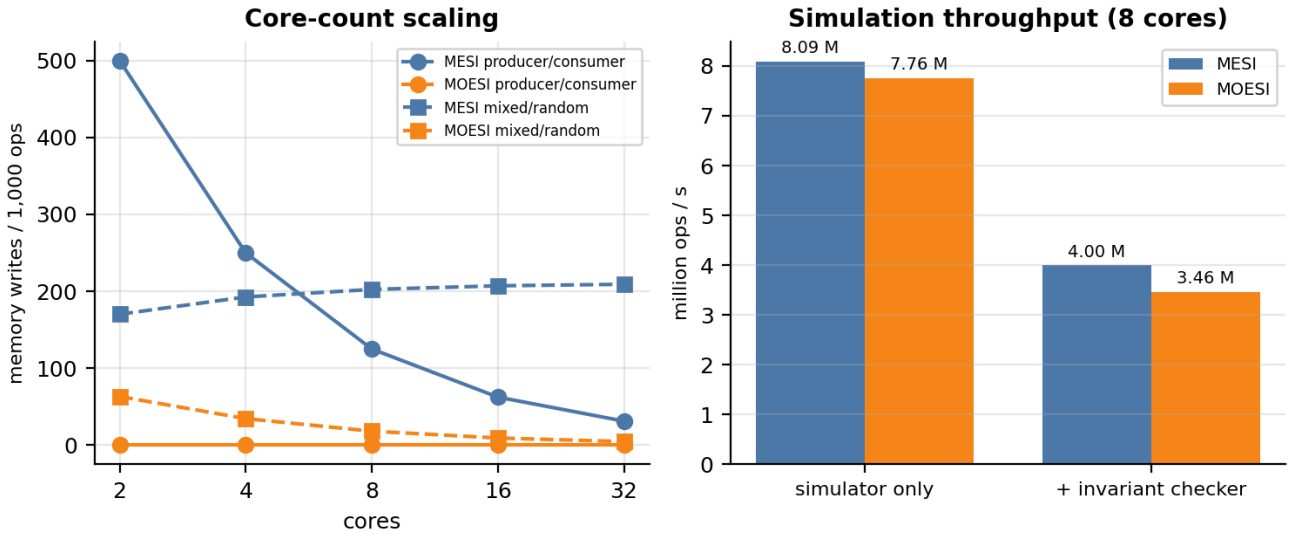


Figure 5. Core-count scaling of memory writes and simulation throughput with and without the checker.

The raw simulator runs at 8.09 million operations per second (MESI) and 7.76 million (MOESI). With the full invariant checker enabled this drops to 4.00 and 3.46 million operations per second, so checking costs about half of the speed.

8.4 Relation to the literature, and limits

The direction of the memory-write result agrees with the literature: Owned-state protocols reduce write-backs, which is the mechanism behind the bandwidth gains reported for MOESIF (Patil 2019) and the case 7 verdict in Table 4. The model does not reproduce the load-miss, latency or power figures of the dual-processor studies (Kaur 2018; Krishna 2025) because it has no timing, power or capacity model; its miss counts are equal for both protocols. The workloads are synthetic patterns chosen to expose protocol differences, caches have unlimited capacity, and all results count events rather than cycles, so they should not be read as performance predictions for real hardware.

9. Conclusion

The evidence supports a simple summary: MESI is the standard four-state baseline, MOESI adds Owned to share dirty data more efficiently, and later variants trade complexity for lower traffic, better predictability, or lower energy.

The executable model supports the state-level claims. All 42 tests pass, including 19 directed state-transition tests, 14 randomized runs of up to 200,000 checked operations and nine injected faults, each of which was caught by the invariant checker. In the benchmarks MOESI eliminates most or all memory write-backs on sharing-heavy workloads at the cost of more cache-to-cache transfers, while hit rate and bus transaction counts are unchanged.

Note that the quantitative results from the literature come from specific studies and configurations, such as one dual-processor simulation and one snooping dual-core design, and should not be assumed to transfer to other systems. Likewise the new benchmark numbers come from a functional, untimed model with synthetic workloads and should be read as event counts.

Appendix A. Code design and working flow

This appendix describes how the C++ model is organized and how an operation flows through it. It replaces the source listing: the design is described by files, data structures, flow diagrams and the points where faults are injected, so that the code can be rebuilt or extended from this description. Section 6 gives the protocol rules and invariants; this appendix shows how the code realizes them.

A.1 Files and roles

File	Role	Main contents
`coherence_verify.cpp`	Model, checker and test suite	`System`, `Harness`, `Stats`, `Bug` enum, the TEST framework, 19 directed tests, randomized stress tests, fault-injection tests, `main()`. Defining `COHERENCE_NO_MAIN` lets another file include it without its `main()`.
`coherence_bench.cpp`	Benchmark driver	Six workload functions, `runOne()`, CSV output, scaling runs and the throughput measurement. It includes the first file and reuses `Harness`.
`plot_bench.py`	Plotting	Reads `bench_results.csv` and draws the benchmark charts.
`bench_results.csv`	Data	One row per workload, protocol and core count (Appendix B).

Table 18. Files that make up the implementation.

A.2 Architecture

The code has three layers. **System** is the protocol model and knows nothing about tests. **Harness** wraps a System, keeps a golden copy of memory and checks every operation. The **test suite** and the **benchmark driver** sit on top and only issue operations through the Harness, so every test and every benchmark run is checked by the same code.

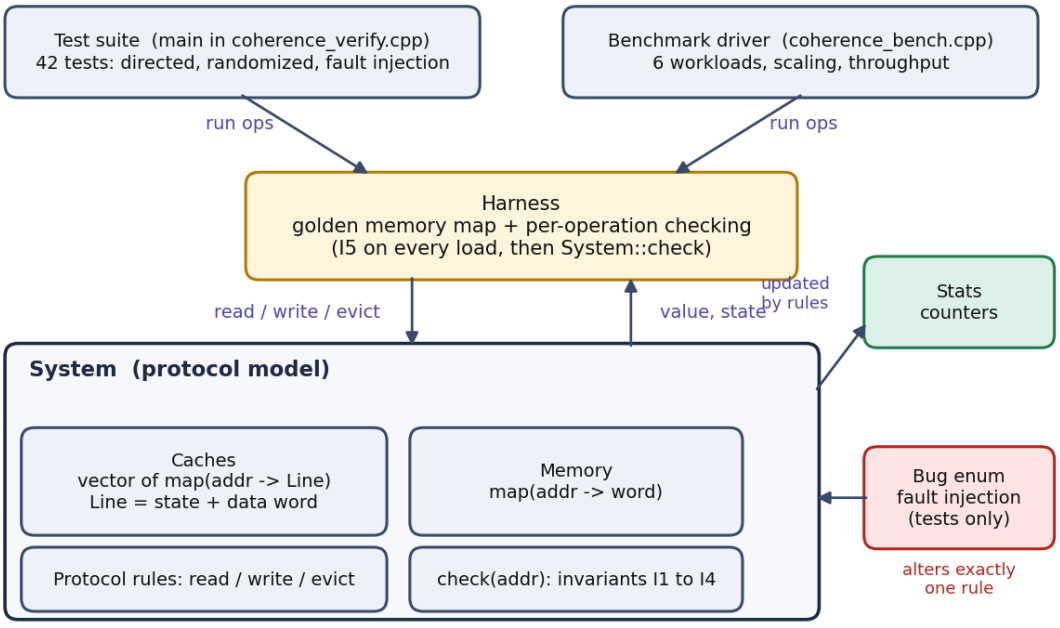


Figure 6. Architecture: test suite and benchmark driver issue operations to the Harness, which drives the System and checks the result.

A.3 Main data structures and functions

Element	Responsibility
`State`	Enum with the five states I, S, E, O, M. A line that is absent from a cache is treated as I.
`Line`	One cache line: a state and one 32-bit data word.
`System` members	A vector of per-core maps from address to Line (the caches), one map from address to word (memory), the protocol (MESI or MOESI), the injected Bug, and a Stats record.
`read / write / evict`	Processor-side operations. Each one performs the bus actions of Table 7 on the other caches directly, because the bus is atomic, and updates the counters.
`invalidateOthers`	Removes the line from every other cache. Used by BusUpgr (write to S or O) and BusRdX (write miss).
`state / cachedData / memData`	Read-only accessors used by the checker and by the tests to inspect the system.
`check(addr)`	Counts the M, O, E and S copies of one address and returns an empty string or a description of the violated invariant (I1 to I4).
`Harness`	Forwards operations, keeps the golden map, applies I5 to every load, runs `check` after every operation, and stores only the first violation.
`Stats`	hits, misses, busRd, busRdX, busUpgr, memWrites, cacheToCache, silentEtoM. These are the quantities reported in Section 8.

Table 19. Data structures and functions of the model.

A.4 Working flow of the three operations

Read. A hit in S, E, O or M returns the cached data with no bus traffic. A miss issues a BusRd, examines every other cache and then installs the line. The remote state decides who supplies the data and how the remote copy changes. This is the only place where MESI and MOESI differ on a read: a remote M copy is flushed to memory and demoted to S in MESI, but demoted to O without touching memory in MOESI.

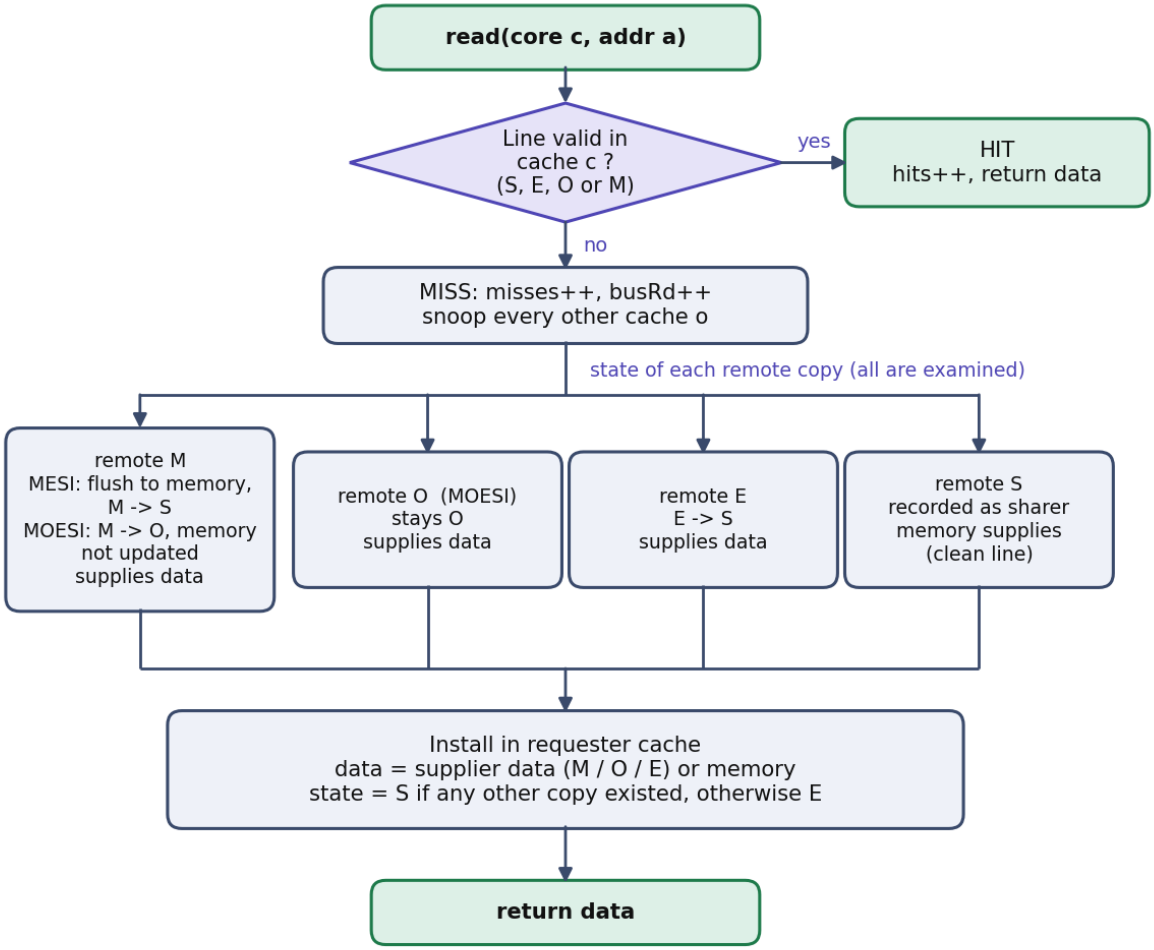


Figure 7. Working flow of read(c, a).

Write. The state of the line in the writing cache selects the action. A write in M is a plain hit, and a write in E upgrades to M silently. A write in S or O needs a BusUpgr, and a write miss needs a BusRdX. Both invalidate every other copy. In all four cases the line ends in state M holding the new value. A remote dirty copy that is dropped by a write miss loses nothing, because the model writes whole lines.

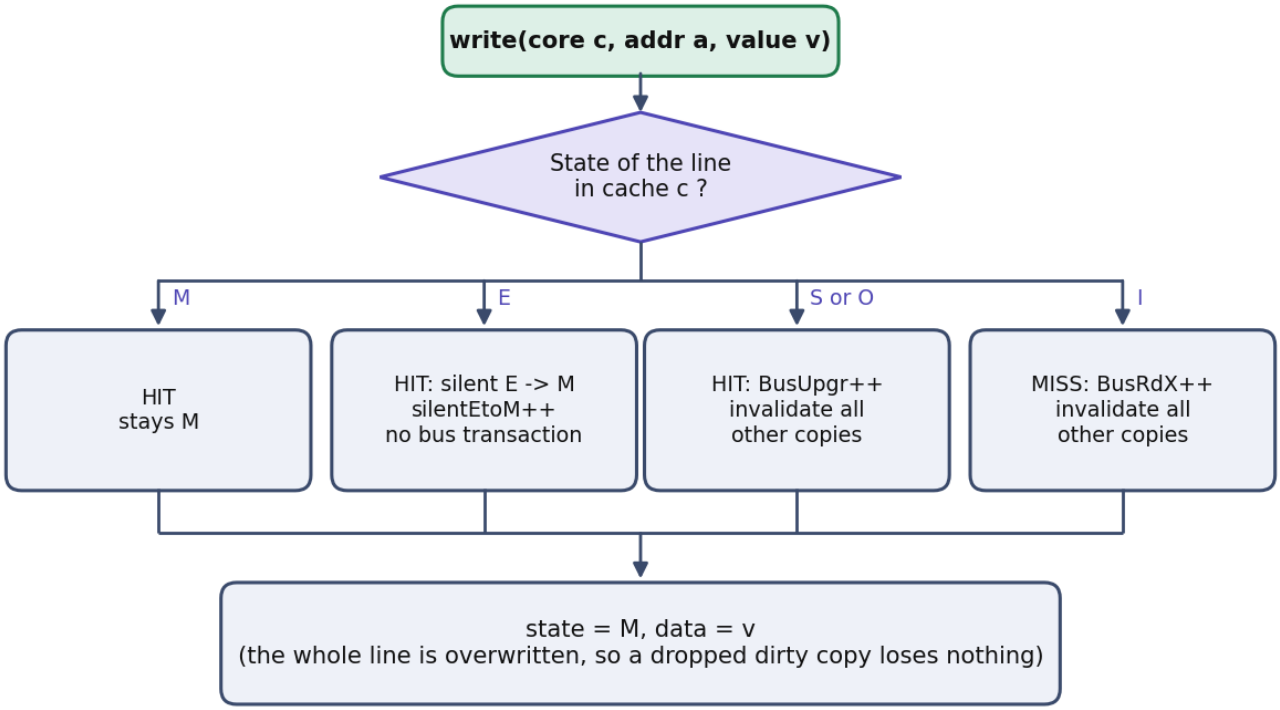


Figure 8. Working flow of write(c, a, v).

Evict. Eviction is the only way a line leaves a cache. A dirty line (M, or O in MOESI) is written back to memory before it is dropped; clean lines (S, E) are dropped silently. Evicting an O line leaves its sharers in S, and they are consistent with memory because of the write-back.

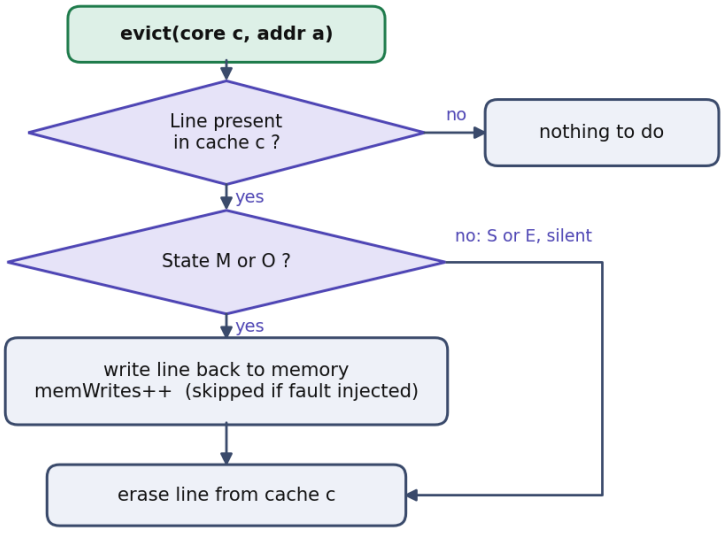


Figure 9. Working flow of evict(c, a).

A.5 Verification flow

Every operation, whether it comes from a directed test, a random run or a benchmark, goes through the same loop. After the System has applied the operation, the Harness compares a load with the golden value (I5) or updates the golden map for a store, and then calls `System::check` for the address that was touched (I1 to I4). The first violation is recorded and the run stops. Checking one address after each operation is sufficient because an operation only changes the state of the one line it names.

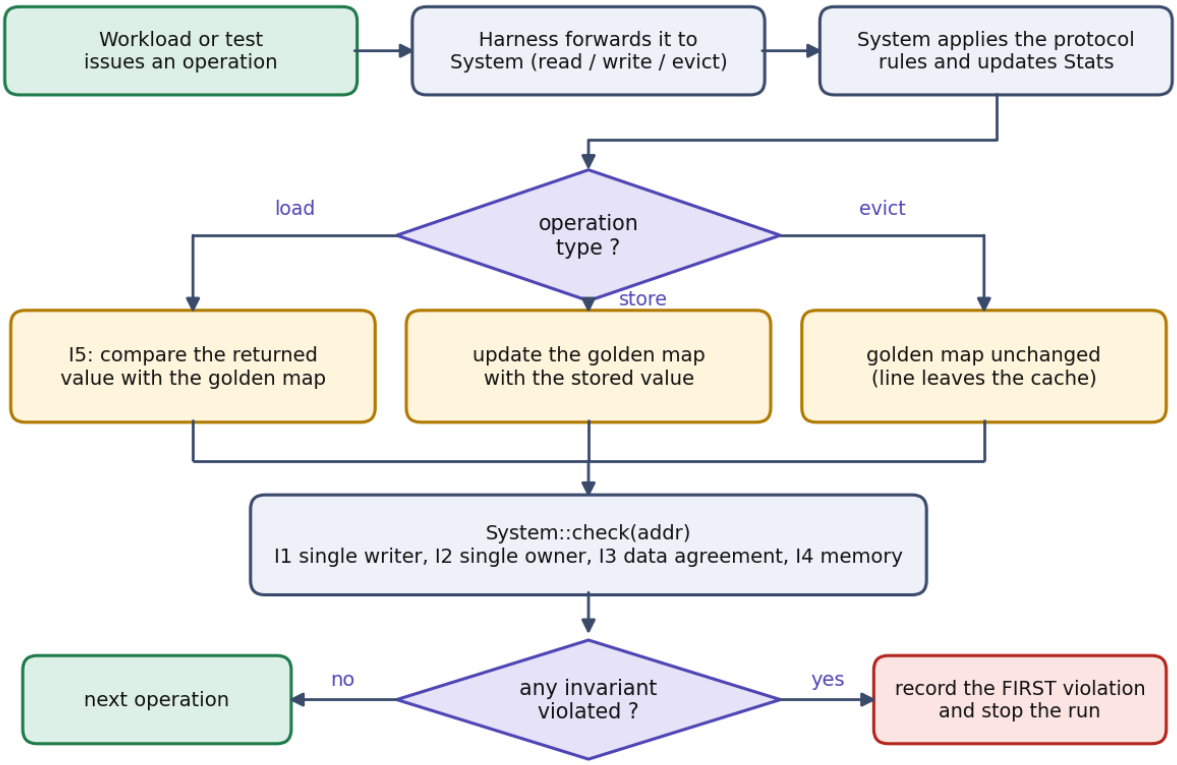


Figure 10. Per-operation verification loop of the Harness.

A.6 Fault-injection hooks

Fault injection reuses the same code path as normal execution. The `Bug` value given to the System constructor disables exactly one rule at one place, and the unchanged checker must then report a violation. With `Bug::None` the model is the correct protocol.

Bug	Place where the rule is disabled	Normal behavior that is skipped
SkipInvalidateOnUpgrade	write, state S or O	Invalidating the other copies on BusUpgr
WriteMissNoInvalidate	write, state I	Invalidating the other copies on BusRdX
NoDowngradeEOnRead	read miss, remote state E	Changing the remote copy from E to S
NoFlushOnMtoS	read miss, remote state M (MESI)	Writing the dirty line back to memory
LostWritebackOnEvict	evict, state M or O	Writing the dirty line back to memory

Table 20. Where each injected fault changes the code path.

A.7 Test and benchmark pipelines

The two programs share the model and the Harness but have different drivers. The test program runs the four test groups, records one PASS or FAIL per test in a results table, prints the list and returns a non-zero exit code if any test fails. The benchmark program runs each workload through `runOne()`, which builds a Harness, runs the workload and aborts with an error if any invariant is violated, then prints one CSV row of counters. The plotting script turns the CSV into charts.

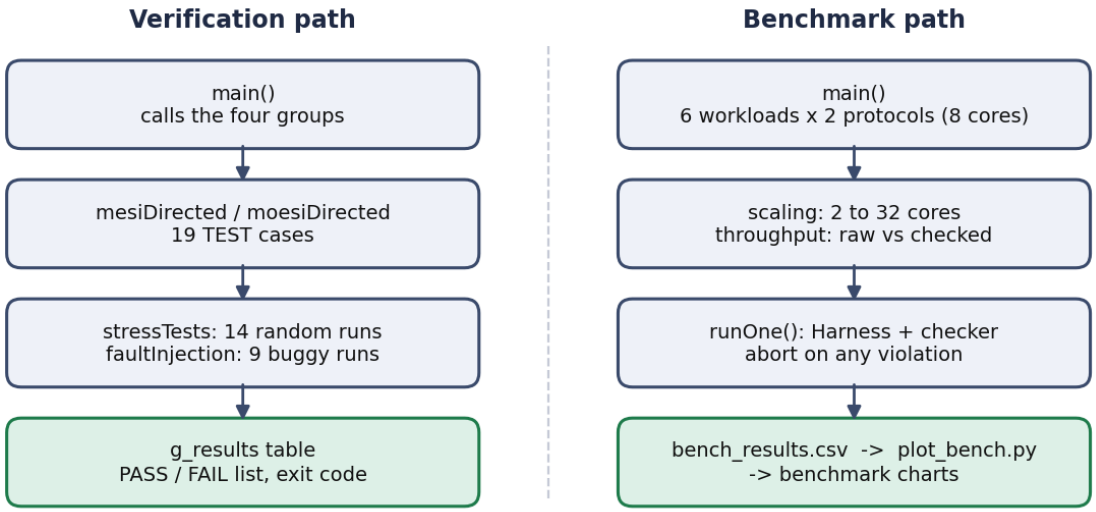


Figure 11. The verification path (tests) and the benchmark path (CSV and charts).

A.8 Design decisions and extension points

Decision	Reason	How to extend
Atomic bus, no transient states	Keeps the model small and makes every operation complete before the next one starts, so the invariants can be checked after each operation.	A split-transaction bus needs transient states and a model checker such as Murphi or TLA+.
One 32-bit word per line, unlimited capacity	Isolates protocol behavior from cache geometry. A write overwrites the whole line, so invalidation can discard remote data.	Add sets, ways and a replacement policy that calls `evict()`; add multi-word lines with partial writes.
Golden model kept outside System	The checker compares against an independent sequential model, so a bug in System cannot hide itself.	Reuse the Harness for any new protocol.
Fault injection through a constructor flag	The same code is exercised with and without the fault, which shows the checker can fail.	Add a Bug value and one conditional at the rule to be broken.
Header-style reuse (`COHERENCE_NO_MAIN`)	The benchmark driver reuses the model and checker without a library build.	Move the model into a header and link the tests and benchmarks against it.
Counters in the model	Section 8 compares protocols by events, which are exact and independent of host timing.	Add a latency table per event to estimate cycles.

Table 21. Main design decisions.

Appendix B. Raw benchmark data

Output of `coherence_bench` (bench_results.csv). Columns: workload, protocol, cores, ops, hits, misses, busRd, busRdX, busUpgr, memWrites, cacheToCache, silentEtoM. For the two throughput rows, the 'hits' column holds operations per second.

```
workload,protocol,cores,ops,hits,misses,busRd,busRdX,busUpgr,memWrites,cacheToCache,silentEtoM
producer_consumer,MESI,8,200000,24999,175001,175000,1,24999,25000,25000,0
producer_consumer,MOESI,8,200000,24999,175001,175000,1,24999,0,175000,0
migratory,MESI,8,200000,100000,100000,100000,0,99999,99999,99999,1
migratory,MOESI,8,200000,100000,100000,100000,0,99999,0,99999,1
read_mostly,MESI,8,200000,147771,52229,49581,2648,7368,9487,9503,0
read_mostly,MOESI,8,200000,147771,52229,49581,2648,7368,0,49495,0
write_contention,MESI,8,200000,24944,175056,0,175056,0,0,0,0
write_contention,MOESI,8,200000,24944,175056,0,175056,0,0,0,0
private_data,MESI,8,200000,199936,64,30,34,0,0,0,30
private_data,MOESI,8,200000,199936,64,30,34,0,0,0,30
mixed_random,MESI,8,200000,38068,131965,69722,62243,12916,40491,39120,65
mixed_random,MOESI,8,200000,38068,131965,69722,62243,12916,3635,66877,65
scale_producer_consumer,MESI,2,200000,99999,100001,100000,1,99999,100000,100000,0
scale_mixed_random,MESI,2,200000,103422,66611,35149,31462,24016,34040,27177,1515
scale_producer_consumer,MOESI,2,200000,99999,100001,100000,1,99999,0,100000,0
scale_mixed_random,MOESI,2,200000,103422,66611,35149,31462,24016,12653,28312,1515
scale_producer_consumer,MESI,4,200000,49999,150001,150000,1,49999,50000,50000,0
scale_mixed_random,MESI,4,200000,65925,104108,54962,49146,20292,38532,35629,286
scale_producer_consumer,MOESI,4,200000,49999,150001,150000,1,49999,0,150000,0
scale_mixed_random,MOESI,4,200000,65925,104108,54962,49146,20292,6905,50562,286
scale_producer_consumer,MESI,8,200000,24999,175001,175000,1,24999,25000,25000,0
scale_mixed_random,MESI,8,200000,38068,131965,69722,62243,12916,40491,39120,65
scale_producer_consumer,MOESI,8,200000,24999,175001,175000,1,24999,0,175000,0
scale_mixed_random,MOESI,8,200000,38068,131965,69722,62243,12916,3635,66877,65
scale_producer_consumer,MESI,16,200000,12499,187501,187500,1,12499,12500,12500,0
scale_mixed_random,MESI,16,200000,20614,149419,78987,70432,7282,41431,40771,10
scale_producer_consumer,MOESI,16,200000,12499,187501,187500,1,12499,0,187500,0
scale_mixed_random,MOESI,16,200000,20614,149419,78987,70432,7282,1861,77348,10
scale_producer_consumer,MESI,32,200000,6249,193751,193750,1,6249,6250,6250,0
scale_mixed_random,MESI,32,200000,10819,159214,84128,75086,3868,41830,41507,3
scale_producer_consumer,MOESI,32,200000,6249,193751,193750,1,6249,0,193750,0
scale_mixed_random,MOESI,32,200000,10819,159214,84128,75086,3868,931,83236,3
throughput_raw,MESI,8,2000000,8089645,0,0,0,0,0,0,0
throughput_checked,MESI,8,2000000,3995198,0,0,0,0,0,0,0
throughput_raw,MOESI,8,2000000,7759069,0,0,0,0,0,0,0
throughput_checked,MOESI,8,2000000,3460296,0,0,0,0,0,0,0
```

References

Alkhamisi 2022. Alkhamisi et al. (2022). Cache coherence issues and solution: a review.
https://consensus.app/papers/performance-comparison-of-cache-coherence-protocol-on-tiwari/ac6dbbf566dd5e989083148bb990d160/

Altwaijry 2013. Altwaijry, Alzahrani (2013). ImprovedMOESI cache coherence protocol.
https://consensus.app/papers/demystifying-cache-coherency-in-modern-multiprocessor-somarouthu/4c6587e7b010539da5c46fd8cbfa74f2/

Amory 2021. Amory, Ahmed (2021). MESI protocol for multicore processors based on FPGA.
https://consensus.app/papers/towards-energy-efficient-approx-cachecoherence-protocol-saraswat-abhishek/397cbac8b7175e9795df8538ff9e6177/

Blanchet 2013. Blanchet, Dupouy (2013). Caches in a multiprocessor environment.
https://consensus.app/papers/moesif-a-mcmp-cache-coherence-protocol-with-improved-patil-mallya/a49f64e60ebf5207861e4802d7b81ceb/

Dey 2014. Dey, Nair (2014). Design and implementation of a simple cache simulator in ....
https://consensus.app/papers/moesil-a-cache-coherency-protocol-for-locked-mixed-nair-pai/db6cfbddbc4d5beb8d1d87425c9e632e/

Faeq 2021. Faeq, Omran (2021). Cache coherency controller for MESI protocol based on FPGA.
https://consensus.app/papers/implementation-and-comparison-of-snoopingbased-cache-krishna-rajeev/bff2c1de7aa556e6b923fe0287c14976/

Fotouhi 2025. Fotouhi et al. (2025). LCSim: a simulation framework for evaluating location https://consensus.app/papers/revisiting-the-complexity-of-hardware-cache-coherence-and-komuravelli-adve/135676ee95a758308e02ec3b7e99c39a/

Ivanov 2001. Ivanov, Nunna (2001). Modeling and verification of cache coherence protocols.
https://consensus.app/papers/an-interactive-mesi-cache-coherence-simulator-for-kehagias-raptis/b206d5fefe3852e3a389b242883730b5/

Kaur 2018. Kaur, Sulochana (2018). Design and implementation of cache coherence protocol for ....
https://consensus.app/papers/designing-predictable-cache-coherence-protocols-for-kaushik-hassan/65824efcd22a5b429dbd50223ba0318b/

Kaushik 2021. Kaushik, Hassan et al. (2021). Designing predictable cache coherence protocols for ....
https://consensus.app/papers/design-and-implementation-of-cache-coherence-protocol-for-kaur-sulochana/0e7639ba73215e6fab68325fa50e140c/

Kehagias 2016. Kehagias, Raptis (2016). An interactive MESI cache coherence simulator.
https://consensus.app/papers/modeling-and-verification-of-cache-coherence-protocols-ivanov-nunna/159706081a9d5a29808d9c687c461ba2/

Komuravelli 2014. Komuravelli, Adve et al. (2014). Revisiting the complexity of hardware cache coherence and ....
https://consensus.app/papers/lcsim-a-simulation-framework-for-evaluating-location-fotouhi/7c02f6b70854553cb5f87f724b586705/

Krishna 2025. Krishna, Rajeev (2025). Implementation and comparison of snooping-based cache ....
https://consensus.app/papers/cache-coherency-controller-for-mesi-protocol-based-on-fpga-faeq-omran/cf4e9517a72d5e1390f79208b107dec1/

Nair 2021. Nair, Pai et al. (2021). MOESIL: a cache coherency protocol for locked mixed ....
https://consensus.app/papers/design-and-implementation-of-a-simple-cache-simulator-in-dey-nair/9ad1a291f1a35f588638e40839e22a92/

Patil 2019. Patil, Mallya et al. (2019). MOESIF: a MCMP cache coherence protocol with improved ....
https://consensus.app/papers/caches-in-a-multiprocessor-environment-blanchet-dupouy/23868ad442f65750be6507ce3dbe1278/

Saraswat 2021. Saraswat, Abhishek et al. (2021). Towards energy efficient approx cache coherence protocol.
https://consensus.app/papers/mesi-protocol-for-multicore-processors-based-on-fpga-amory-ahmed/429075dba89b51019d0b694dad1a19ad/

Somarouthu 2025. Somarouthu (2025). Demystifying cache coherency in modern multiprocessor ....
https://consensus.app/papers/improvedmoesi-cache-coherence-protocol-altwaijry-alzahrani/f797d13583065bb5b853b36b214f8af9/

Tiwari 2014. Tiwari et al. (2014). Performance comparison of cache coherence protocol on ....
https://consensus.app/papers/cache-coherence-issues-and-solution-a-review-alkhamisi/c112264c03955e3a8671c252c76448db/

Titles ending in an ellipsis were truncated in the source links and are shown as available.